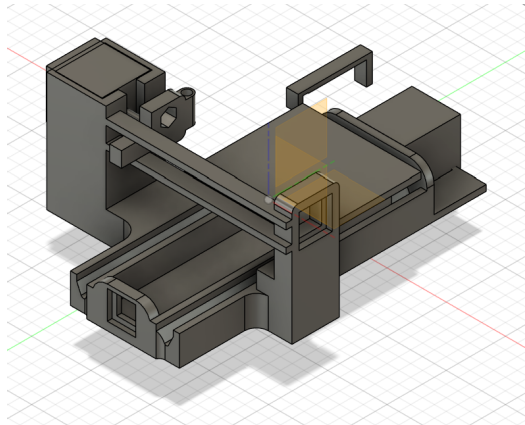


MAE 494/598: Mechatronics Engineering for Design and
Manufacturing
Spring 2026

Design and Development of a Desktop Laser Engraving System

Application Project Report



Team 8

Nithesh Raj Ashok, Nico Carson, Amruth Parimi, Manoj Kaipu,
Dilli Babu Kalluru, Erik Wisnom

Arizona State University
May 2026

Contents

1	Objective of the Application Project	3
2	Basic Information of the Building Product / Functional Module	4
2.1	Product Definition	4
2.2	Primary Functional Requirements	4
2.3	Design Scope	5
3	Basic Construction and Features	5
3.1	Mechanical Construction	5
3.2	Electrical Construction	6
3.3	Visible Prototype Features	7
3.4	Key Features	8
4	Mechatronics System Description	9
4.1	System Architecture	9
4.2	Open-Loop Position Control	9
4.3	Manual Control Mode	10
4.4	Automated Control Mode	10
5	Sub-Functional Components / Modules	11
5.1	Mechanical Frame Module	11
5.2	X-Axis Motion Module	11
5.3	Y-Axis Motion Module	11
5.4	Laser Module	11
5.5	User Input Module	12
5.6	Display Module	12

5.7	Embedded Software Module	12
6	Electronic, Control, and Mechanical Systems	13
6.1	Electronic Systems and Components	13
6.1.1	Arduino UNO Controller	13
6.1.2	Stepper Motor Drivers	13
6.1.3	Laser Output Circuit	13
6.1.4	LCD and Human-Machine Interface	13
6.2	Control Systems and Components	14
6.2.1	Manual Control Logic	14
6.2.2	Reset / Homing Logic	14
6.2.3	Automated Shape Control	14
6.3	Mechanical Systems and Components	15
6.3.1	Lead Screw Motion Conversion	15
6.3.2	Structural Rigidity and Alignment	15
6.3.3	Work Envelope	15
7	Testing and Results	16
8	Thought Evaluation and Improvements	17
8.1	Evaluation of the Current Design	17
8.2	Recommended Improvements	18
9	Conclusion and Summary	19
10	Team Member Contribution Table	20
A	Manual Control Arduino Code	21
B	Automated Control Arduino Code	25

1. Objective of the Application Project

The objective of this project was to design, build, program, and test a compact desktop two-axis laser engraving system. The system was intended to demonstrate the integration of mechanical motion, electronics, embedded control, and user interaction in a functional mechatronic product. The application target was low-power laser marking and engraving on lightweight materials such as cardboard, wood, and coated surfaces.

The project goals were organized around five engineering outcomes:

1. Develop a desktop XY gantry platform capable of repeatable planar positioning.
2. Integrate two stepper-motor-driven lead-screw axes with a laser module mounted to the moving carriage.
3. Implement manual joystick-based control with live coordinate feedback on a 16-by-2 LCD.
4. Implement an automated-control mode capable of selecting a predefined shape and executing a toolpath.
5. Identify mechanical, electrical, and software limitations and propose realistic improvements for the next design iteration.

Laser engraving is appropriate for this project because it is a non-contact material-removal process: energy from a focused laser beam removes or modifies surface material without a conventional cutting tool. This avoids tool wear and enables high geometric flexibility, but it also makes motion accuracy, laser focusing, electrical stability, and safety critical design concerns [1].

2. Basic Information of the Building Product / Functional Module

The developed product is a small-scale desktop laser engraving module based on a two-degree-of-freedom Cartesian gantry. The machine converts user commands or stored toolpath commands into synchronized motion of an X axis and a Y axis while switching a laser module on or off at the required locations. Its structure is comparable to a simplified CNC plotter or laser engraver, but with a custom embedded control program rather than a full commercial CNC controller.

2.1. Product Definition

The system is a functional desktop XY laser engraver. The X axis moves the laser carriage, while the Y axis moves the work surface or sled. The laser module is mounted above the workpiece and engraves by exposing the material surface along a programmed or manually traced path. The controller tracks position using commanded step counts rather than external encoder feedback.

2.2. Primary Functional Requirements

Table 1: Primary functional requirements of the desktop laser engraving system.

Requirement	Implementation in the project
Two-axis planar motion	Two stepper motors drive lead screws to create X and Y translation.
Manual operation	A thumbstick joystick provides analog X and Y commands. The joystick pushbutton toggles the laser.
Position display	A 16-by-2 LCD displays the estimated X and Y coordinates in millimeters.
Laser actuation	A digital output controls the laser module. The design also allows future PWM intensity control.
Reset / homing behavior	A reset button commands both axes back to the stored origin by reversing the accumulated step count.
Automated operation	A menu-driven program allows shape selection; the implemented routine draws a square using relative motion commands.
Compact construction	The frame uses 3D-printed structural parts and screw-driven linear motion to keep cost and size low.

2.3. Design Scope

The system was designed as a proof-of-concept educational mechatronics product rather than an industrial laser engraver. The core emphasis was the integration of functional subsystems: mechanical structure, motor actuation, embedded software, laser switching, user input, and display feedback. Industrial features such as certified laser enclosure, interlocks, exhaust ventilation, closed-loop feedback, and G-code compatibility were identified as future improvements rather than initial requirements.

3. Basic Construction and Features

3.1. Mechanical Construction

The mechanical structure uses a compact 3D-printed frame, two lead screws, motor mounts, a base plate, and a laser carriage. The gantry geometry is Cartesian: one axis controls horizontal laser positioning while the other controls workpiece motion. This architecture is simple, low-cost, and mechanically intuitive because each motor maps primarily to one coordinate direction.

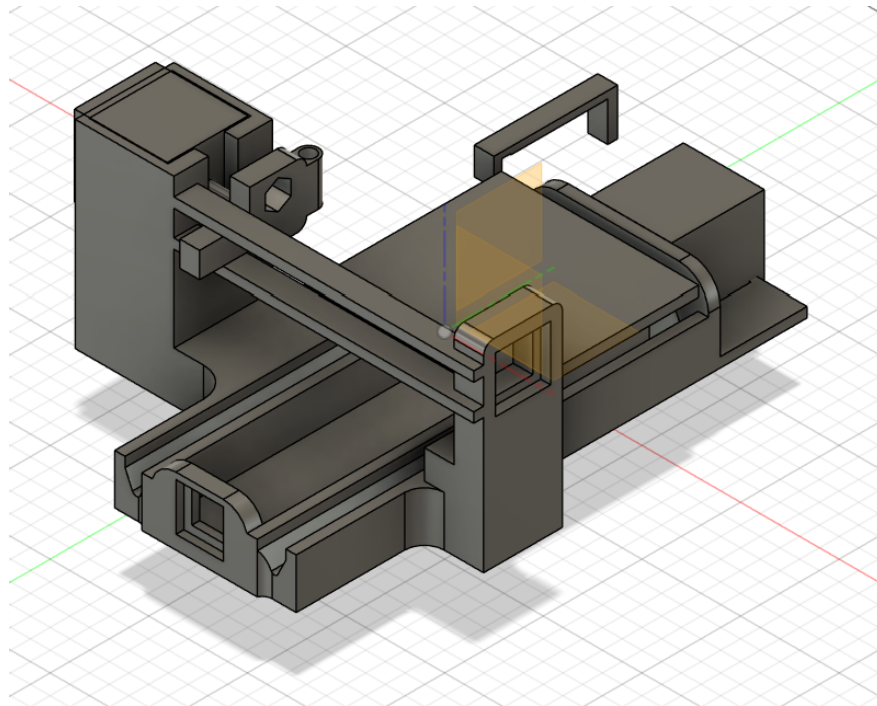


Figure 1: CAD model of the desktop XY gantry and laser carriage assembly.

Lead screws were selected because they provide high mechanical advantage and convert rotary motor motion into linear translation. Compared with belt drives, lead screws generally support better static positioning and higher resistance to back-driving, but they can also introduce friction, backlash, and alignment sensitivity. These tradeoffs were directly observed during testing: the system achieved controlled motion but required speed reductions in the automated code to overcome friction reliably.

3.2. Electrical Construction

The electrical architecture is built around an Arduino UNO, two motor drivers, two stepper motors, a laser module, a joystick module, a reset button, and an LCD. The Arduino UNO is suitable for this scale of system because it provides digital I/O, analog inputs, and PWM-capable pins on an ATmega328P microcontroller platform [2]. The LiquidCrystal library supports character LCDs based on the common HD44780-compatible interface, which matches the coordinate display used in the system [4].

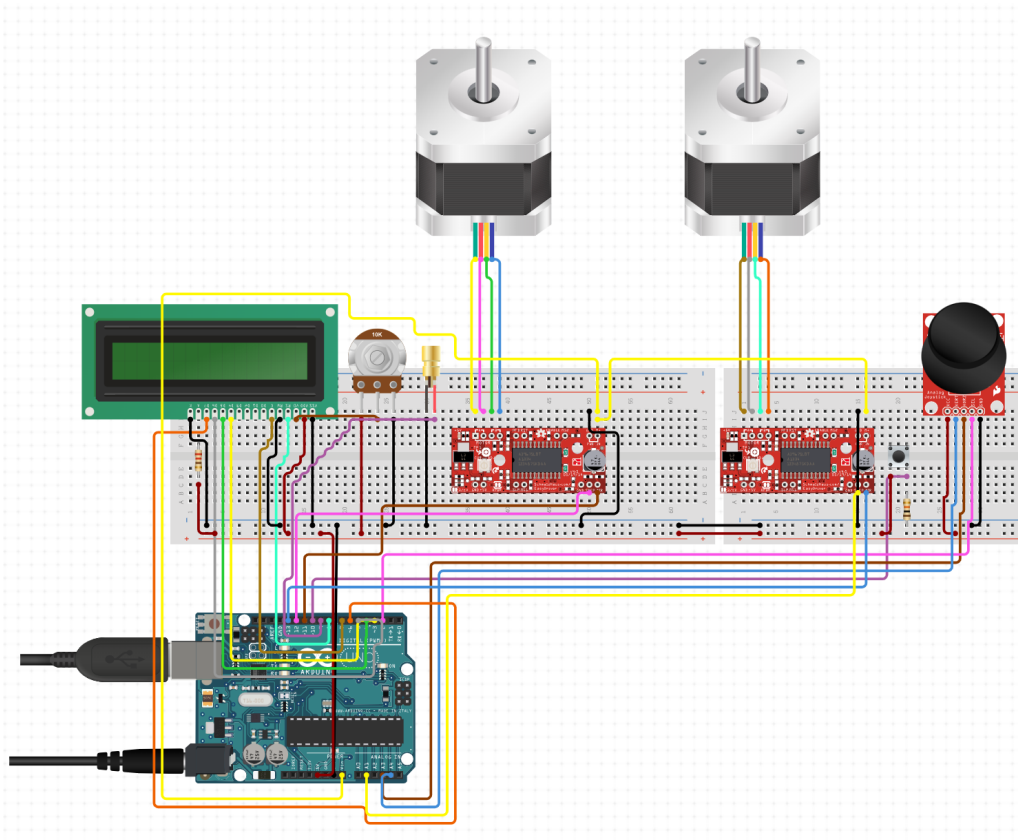
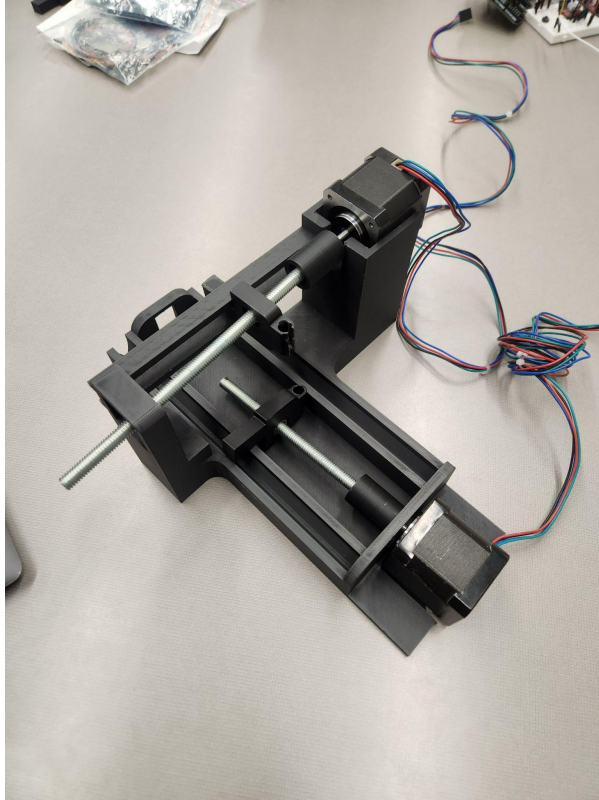


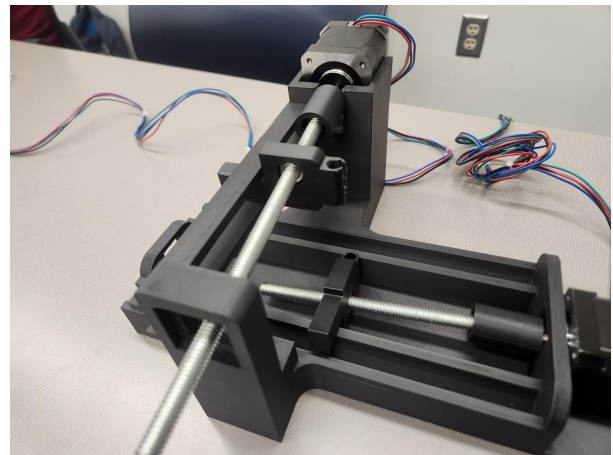
Figure 2: System wiring diagram showing Arduino, LCD, joystick, motor drivers, stepper motors, reset button, and laser output.

3.3. Visible Prototype Features

The physical prototype integrates the printed frame, linear motion components, motors, and electronics into a working benchtop platform. The pictures below show the mechanical assembly and control wiring used during integration and testing.



(a) Top view of the assembled axis components.



(b) Axis assembly showing lead screw, motor, and carriage alignment.

Figure 3: Physical construction of the XY motion platform.

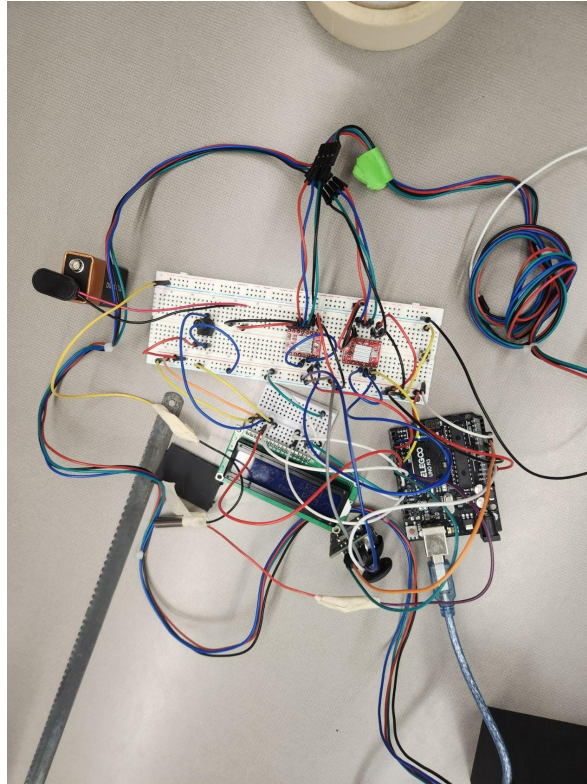


Figure 4: Prototype electronics wiring during integration. The breadboard wiring approach worked for testing but is not mechanically robust for long-term operation.

3.4. Key Features

The final prototype includes the following features:

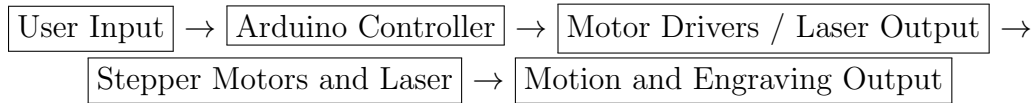
- Two-axis lead-screw-driven motion.
- Manual joystick control with analog speed mapping and deadband filtering.
- Laser toggle using the joystick pushbutton.
- LCD coordinate display in millimeters.
- Software-defined travel limits using step-count boundaries.
- Reset button that returns the system to the origin based on accumulated step counts.
- Automated square-drawing routine using relative motion commands.
- Menu structure prepared for additional shapes: square, hexagon, triangle, circle, and star.

4. Mechatronics System Description

The project is a mechatronic system because its performance depends on the coordinated behavior of mechanical hardware, electrical circuits, embedded control, and human-machine interaction. The system does not merely contain electronics attached to a mechanism; rather, the mechanism is commanded, constrained, and interpreted through software.

4.1. System Architecture

The system-level control flow is:



The Arduino receives analog joystick values, pushbutton states, and reset-button input. It computes axis direction, pulse timing, laser state, and display values. Stepper motor drivers translate step and direction signals into phase currents for the motors. The motors rotate the lead screws, converting electrical pulses into linear carriage movement. The laser marks the workpiece when enabled.

4.2. Open-Loop Position Control

The prototype uses open-loop stepper control. In open-loop operation, the controller assumes that each commanded pulse produces one valid motor step. The Arduino maintains internal counters for the X and Y axes. These counters are converted to position estimates by:

$$x_{mm} = \frac{N_x}{S_{mm}}, \quad y_{mm} = \frac{N_y}{S_{mm}} \quad (1)$$

where N_x and N_y are accumulated step counts and $S_{mm} = 141.74$ steps/mm is the software conversion factor. The corresponding travel limits are:

$$X_{max} = \frac{18000}{141.74} \approx 127.0 \text{ mm} \quad (2)$$

$$Y_{max} = \frac{12000}{141.74} \approx 84.7 \text{ mm} \quad (3)$$

This control method is simple and computationally efficient, but it does not detect missed steps, mechanical slip, backlash, or motor stalls. Stepper motors are commonly used in position-control applications because they can be commanded to move in discrete increments, but reliable open-loop accuracy depends on the load remaining within the motor-driver torque envelope [3].

4.3. Manual Control Mode

In manual mode, the thumbstick analog voltages are read through Arduino analog inputs A0 and A1. The software defines a deadzone between 490 and 530 ADC counts to prevent drift around the joystick center. When the joystick is moved outside the deadzone, the controller sets the appropriate motor direction and generates step pulses.

The step delay is mapped from joystick deflection. Larger deflection produces a shorter delay between steps, which increases the motor stepping rate. The X axis uses a faster speed range than the Y axis, reflecting differences in friction, torque demand, or observed mechanical response. The laser is toggled using the joystick pushbutton, and a 200 ms delay is used as a simple debounce interval.

4.4. Automated Control Mode

The automated program introduces a simple menu state machine. The LCD displays a shape selection menu, and the joystick X axis scrolls through available shapes. The square routine is implemented and executed using relative motion commands. The software moves to an offset point, turns the laser on, traces a one-inch square, turns the laser off, and returns home.

The automated control routine uses slower speeds than the manual program. This was a necessary design correction because the lead-screw axes experienced friction and torque limitations. The adjusted pulse delays improve reliability by giving the motors more time to develop torque and settle between steps.

5. Sub-Functional Components / Modules

5.1. Mechanical Frame Module

The frame provides the datum structure for the axes. Its functions are to support the motors, constrain the lead screws, hold the laser carriage, and maintain relative alignment between the laser and workpiece. The use of 3D-printed parts reduced cost and allowed rapid iteration; however, printed polymer structures are less stiff than metal extrusions or machined plates. This limitation contributed to vibration and alignment sensitivity.

5.2. X-Axis Motion Module

The X-axis module moves the laser carriage along one horizontal direction. It includes the stepper motor, motor mount, lead screw, carriage, and guiding structure. In the manual-control code, the X axis is assigned to direction pin 2 and step pin 3. In the automated-control code, the pin assignment is swapped so that X uses direction pin 5 and step pin 6. This means the physical wiring and code must be matched carefully before operation.

5.3. Y-Axis Motion Module

The Y-axis module moves the sled or workpiece support. It follows the same basic architecture as the X axis but requires separate calibration because the moving load, screw friction, and alignment conditions are not identical. The software intentionally uses a slower Y-axis pulse delay range. In automated control, $autoSpeedY = 3500$ microseconds, while $autoSpeedX = 3000$ microseconds.

5.4. Laser Module

The laser module is the energy-delivery element. Its function is to heat, darken, melt, or vaporize the surface layer of the workpiece along the commanded path. The prototype controls the laser through a digital output on pin 7. Although the presentation of the system identifies PWM laser control as a capability, the supplied code primarily uses binary on/off switching. True PWM intensity modulation would require using a PWM-capable Arduino output pin and an appropriate driver stage for the laser input.

This distinction matters: on/off control is enough for simple marking and path verification, but PWM control is necessary for grayscale engraving, power compensation, and feed-rate-dependent burn correction.

5.5. User Input Module

The joystick module provides two analog axes and a pushbutton. The analog axes control motion or menu selection, depending on the software mode. The joystick button toggles the laser in manual mode or confirms a shape selection in automated mode. The reset button is a separate safety and usability input that commands return-to-home behavior.

5.6. Display Module

The 16-by-2 LCD gives immediate local feedback. In manual mode, it displays estimated X and Y position in millimeters. In automated mode, it displays the shape-selection menu and drawing/homing status messages. The display prevents the project from being a black-box controller and improves debugging because motor motion can be compared with expected coordinates.

5.7. Embedded Software Module

The software performs five main functions:

1. Read user inputs from analog and digital pins.
2. Convert joystick values into direction and step timing.
3. Generate step pulses for the motor drivers.
4. Track position through accumulated step counts.
5. Update the LCD and manage laser state.

The automated program adds a state machine and a relative-motion function. This is a more scalable structure than directly writing individual step commands throughout the loop because it separates high-level path commands from low-level pulse generation.

6. Electronic, Control, and Mechanical Systems

6.1. Electronic Systems and Components

6.1.1. *Arduino UNO Controller*

The Arduino UNO acts as the central controller. It reads the joystick and buttons, sends step and direction commands, switches the laser output, and updates the LCD. The UNO provides 14 digital input/output pins, 6 analog inputs, and 6 PWM-capable digital outputs, which is sufficient for this prototype [2].

6.1.2. *Stepper Motor Drivers*

The motor drivers form the power-electronics interface between low-current Arduino logic signals and the higher-current stepper motors. A typical driver for this type of prototype is the A4988 or a similar step/direction driver. The A4988 supports microstepping, adjustable current limiting, and protection features such as over-current and over-temperature protection [5]. These features are important because stepper motor torque, heat generation, and missed-step probability depend strongly on driver current configuration.

6.1.3. *Laser Output Circuit*

The software assigns the laser to digital pin 7. In a production-quality circuit, the Arduino pin should not directly power a laser diode. Instead, it should drive a transistor, MOSFET, or dedicated laser driver input. This protects the microcontroller and allows controlled current delivery to the laser module. If PWM power control is implemented, the output should be moved to a PWM-capable pin such as 3, 5, 6, 9, 10, or 11 on the UNO, depending on timer conflicts and wiring constraints.

6.1.4. *LCD and Human-Machine Interface*

The LCD uses the LiquidCrystal library and a parallel interface on pins 8 through 13. This display is simple but effective for an embedded product because it provides local status without requiring a PC connection. It displays live coordinates in manual mode and menu

states in automated mode.

6.2. Control Systems and Components

6.2.1. Manual Control Logic

Manual control is implemented as a polling loop. The controller reads the joystick, compares each axis against a deadzone, sets direction, produces one step, and updates the software position counter. The loop then updates the LCD at a 250 ms interval. The LCD update is non-blocking because it is scheduled using `millis()` rather than constant clearing or printing on every loop iteration.

A strength of this design is its simplicity: the code is easy to debug and maps directly to observed hardware behavior. A weakness is that step generation still uses blocking `delayMicroseconds()` calls, so simultaneous multi-axis interpolation is approximate rather than mathematically coordinated.

6.2.2. Reset / Homing Logic

The reset button calls `returnToHome()`. This function turns the laser off, displays homing status, sets both motor directions backward, and steps each axis until its internal position counter returns to zero. This is not true sensor-based homing. It is a software return-to-origin based on remembered step counts.

The distinction is important. If the machine misses steps, stalls, or is moved manually while powered off, the internal counter becomes inaccurate. A better design would use physical limit switches, optical interrupters, or Hall-effect sensors to establish a repeatable machine coordinate origin.

6.2.3. Automated Shape Control

The automated code is a first step toward CNC-style control. It introduces:

- A `MENU` state for shape selection.
- A `DRAWING` state for path execution.

- A reusable `moveRelative(deltaX, deltaY)` function.
- A predefined square toolpath based on one-inch side lengths.

This structure can be extended to additional shapes by writing path-generation routines for each menu option. For example, a circle could be approximated by many small linear segments, while a star could be implemented as a sequence of polar-coordinate vertex moves.

6.3. Mechanical Systems and Components

6.3.1. Lead Screw Motion Conversion

The lead screws convert motor rotation into linear displacement. The software calibration constant of 141.74 steps/mm combines motor step angle, microstepping configuration, and screw pitch. This constant is central to coordinate accuracy. If the driver microstep mode or screw pitch changes, the conversion must be recalibrated.

6.3.2. Structural Rigidity and Alignment

The machine experienced screw alignment, vibration, and calibration challenges. These are expected failure modes in low-cost gantry systems. Misalignment increases friction and can cause lost steps. Low frame stiffness increases vibration and reduces engraving quality. Backlash can make dimensions inconsistent when direction reverses.

6.3.3. Work Envelope

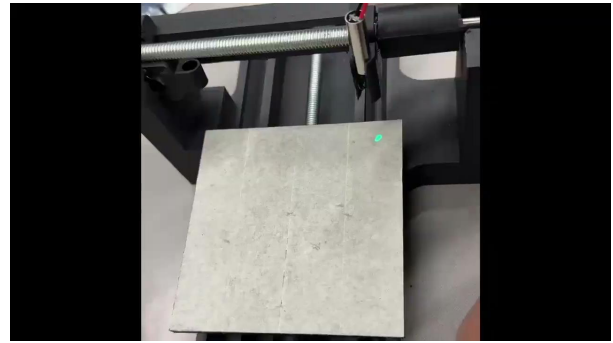
Based on the software step limits and calibration factor, the programmed work envelope is approximately 127.0 mm in X and 84.7 mm in Y. This is sufficient for small test coupons and classroom demonstrations. A larger work area would require longer guide elements, better frame stiffness, cable routing, and more robust motor torque margins.

7. Testing and Results

Testing focused on functional verification rather than formal metrology. The system was evaluated by checking axis motion, joystick response, laser toggling, coordinate display updates, reset behavior, and basic engraving output.



(a) Motion and laser positioning test.



(b) Engraving demonstration on a work surface.

Figure 5: Demonstration stills from system testing.



Figure 6: Representative engraving-related sample image used to frame the target application.

The prototype demonstrated successful integration of motion and laser control. Manual joystick operation produced controlled axis movement. The LCD provided readable coordinate feedback. The reset function returned the system to its software-defined origin, and the automated square routine established a foundation for shape-based engraving.

The main limitations observed during testing were:

- Mechanical friction in the lead-screw system.
- Vibration from the printed frame and unsupported wiring.
- Screw alignment sensitivity.
- Inconsistent electrical connections from breadboard wiring.
- Calibration dependence on the assumed steps/mm value.
- Lack of true homing sensors.
- Lack of a laser enclosure and emergency-stop interlock.

8. Thought Evaluation and Improvements

8.1. Evaluation of the Current Design

The design succeeds as a mechatronics integration project. It demonstrates the key engineering relationships between actuation, sensing/input, embedded computation, user interface, and mechanical output. The strongest part of the design is the clear mapping between commands and physical behavior: joystick deflection produces motion, step counts produce displayed position, and automated relative moves produce geometric paths.

However, the current implementation is not yet a robust engraving product. The most significant limitation is that position is estimated only from commanded steps. There is no feedback confirming that the axes actually moved. This is acceptable for a classroom prototype but not enough for reliable repeated engraving. The second major limitation is electrical robustness. Breadboard wiring and loose connections are common sources of intermittent faults in systems with moving wires and motor current spikes. The third major limitation is laser safety. Any laser engraver should eventually include an enclosure, interlock, emergency stop, eye-safety precautions, and fume control.

8.2. Recommended Improvements

Table 2: Recommended improvements for the next prototype iteration.

Area	Problem in current design	Recommended correction
Electrical wiring	Breadboard wiring is fragile and vulnerable to vibration.	Replace with a CNC shield, soldered perfboard, or custom PCB. Add strain relief and connectors.
Homing	Reset is based on software step counters only.	Add limit switches or optical sensors for true homing and machine-coordinate recovery.
Laser control	Current code uses binary laser on/off control.	Use a PWM-capable pin and a proper laser driver for controlled intensity.
Safety	No enclosure, interlock, or emergency-stop chain is included.	Add enclosed housing, keyed enable, door interlock, emergency stop, and laser warning labeling.
Mechanical stiffness	Printed parts and screw alignment create vibration and friction.	Use linear rails, stiffer frame members, improved bearing support, and better screw couplers.
Calibration	Steps/mm is hard-coded and not experimentally validated in the report.	Measure actual displacement using calipers, calculate error, and update calibration constants.

Area	Problem in current design	Recommended correction
Software scalability	Automated mode currently executes only square drawing.	Add general path planning, acceleration profiles, and G-code support. GRBL is a realistic future option because it provides open-source embedded G-code motion control for Arduino-class CNC machines [6].
Motion quality	Blocking delays limit smooth coordinated motion.	Use timer interrupts or a motion-planning library with acceleration and Bresenham-style interpolation.
Thermal/fume control	Engraving can generate smoke and deposits.	Add ventilation, air assist, and material-specific operating procedures.

9. Conclusion and Summary

This project produced a functioning desktop laser engraving system built around a two-axis lead-screw gantry, stepper motors, an Arduino UNO controller, joystick input, LCD feedback, and laser actuation. The system met the central educational objective: it demonstrated mechatronic integration through the combined operation of mechanical positioning, electrical drive circuitry, embedded control logic, and user interaction.

The manual-control program provides intuitive joystick operation with coordinate feedback and reset-to-origin functionality. The automated-control program extends the design by adding a menu-driven state machine and a predefined square toolpath. Together, these two programs show a progression from direct human control to programmed toolpath execution.

The prototype also exposed realistic engineering constraints. Mechanical alignment, vibration, friction, wiring stability, motor calibration, and laser safety were not secondary details; they directly affected system performance. The strongest future improvements are the addition of

real homing sensors, a rigid frame, a safer laser enclosure, a stable driver board, calibrated motion parameters, and a GRBL- or G-code-based toolpath pipeline.

Overall, the project was successful as a desktop mechatronics demonstrator. It provides a sound foundation for a more capable, safer, and more repeatable laser engraving system.

10. Team Member Contribution Table

Team Member	Contribution Description	Estimated Contribution
Nithesh Raj Ashok	Presenting, Report Preparation, CAD Design & Mechanical Assembly. Co-presented the final project and authored the mechanical design section. Led Solid-Works modeling of the XY gantry frame, motor mounts, and laser carriage. Assembled 3D-printed components and verified axis alignment.	16.7%
Nico Carson	Presenting, Report Preparation, Software Integration & Computation Testing. Co-presented system operation and authored the embedded control section of the report. Integrated and debugged both Arduino programs. Verified step-count-to-millimeter calibration constants and validated work envelope limits numerically.	16.7%
Amruth Parimi	Presenting, Report Preparation, Electromechanical Troubleshooting & System Testing. Participated in the final presentation and co-authored the testing and results section. Resolved motor driver faults and lead-screw misalignment. Ran functional verification tests across axis motion, laser toggling, and reset behavior.	16.7%
Manoj Kaipu	Presenting, Report Preparation, Embedded Software Development & System Architecture. Co-presented and authored the system description and sub-module sections. Developed the menu state machine and <code>moveRelative()</code> path function. Defined the overall mechatronic control flow and module structure.	16.7%
Dilli Babu Kalluru	Presenting, Report Preparation, Electrical Wiring & Component Sourcing. Participated in the final presentation and documented the electrical architecture in the report. Built the full breadboard circuit for the Arduino, motor drivers, LCD, joystick, and laser module. Researched and procured all hardware within budget.	16.7%
Erik Wisnom	Presenting, Report Preparation, Hardware Prototyping & Design Evaluation. Co-presented the live engraving demonstration and authored the evaluation and improvements section. Assembled and iterated on the physical prototype. Identified mechanical limitations and proposed corrective improvements for the next design iteration.	16.7%

A. Manual Control Arduino Code

```
1 #include <LiquidCrystal.h> // Initialize the standard LCD library with your pins
2 LiquidCrystal lcd(8, 9, 10, 11, 12, 13);
3
4 // Motor 1 (X-Axis)
5 const int dirPin1 = 2;
6 const int stepPin1 = 3;
7 const int joyX = A0;
8
9 // Motor 2 (Y-Axis)
10 const int dirPin2 = 5;
11 const int stepPin2 = 6;
12 const int joyY = A1;
13
14 // Lasers and Switches
15 const int laserPin = 7;
16 const int joySW = 4;
17 const int resetBtnPin = A2; // Reset Button Pin
18
19 // Variables for Laser Toggle Logic
20 bool laserState = LOW;
21 bool lastButtonState = HIGH;
22
23 // Adjust these for motor sensitivity
24 const int deadzoneLow = 490;
25 const int deadzoneHigh = 530;
26
27 // --- Boundary Variables ---
28 const long maxXSteps = 18000;
29 const long maxYSteps = 12000;
30
31 // Running tally of current positions
32 long currentXSteps = 0;
33 long currentYSteps = 0;
34
35 // Conversion factor for display
36 const float stepsPerMM = 141.74;
37
38 // LCD Update Timer Variables
39 unsigned long lastLCDUpdate = 0;
40 const int lcdInterval = 250;
41
42 void setup() {
43     // Set up the LCD
44     lcd.begin(16, 2);
45     lcd.setCursor(0, 0);
46     lcd.print("X: 0.0 mm");
47     lcd.setCursor(0, 1);
48     lcd.print("Y: 0.0 mm");
```

```
49
50 pinMode(dirPin1, OUTPUT);
51 pinMode(stepPin1, OUTPUT);
52 pinMode(dirPin2, OUTPUT);
53 pinMode(stepPin2, OUTPUT);
54 pinMode(laserPin, OUTPUT);
55 pinMode(joySW, INPUT_PULLUP);
56 // Enable internal pull-up resistor for the reset button
57 pinMode(resetBtnPin, INPUT_PULLUP);
58 }
59
60 void loop() {
61     // --- Check Reset Button FIRST ---
62     // If pressed (goes LOW), trigger the homing sequence
63     if (digitalRead(resetBtnPin) == LOW) {
64         returnToHome();
65     }
66
67     // --- Laser Toggle Logic ---
68     bool currentButtonState = digitalRead(joySW);
69     if (currentButtonState == LOW && lastButtonState == HIGH) {
70         laserState = !laserState;
71         digitalWrite(laserPin, laserState);
72         delay(200);
73     }
74     lastButtonState = currentButtonState;
75
76     // --- Read Joystick Axes ---
77     int xVal = analogRead(joyX);
78     int yVal = analogRead(joyY);
79
80     // --- Process Motor 1 (X-Axis) ---
81     // FAST PACE: Delay mapped from 2000 (slowest) to 800 (fastest)
82     if (xVal > deadzoneHigh && currentXSteps < maxXSteps) {
83         digitalWrite(dirPin1, HIGH);
84         stepMotor(stepPin1, map(xVal, 531, 1023, 2000, 800));
85         currentXSteps++;
86     } else if (xVal < deadzoneLow && currentXSteps > 0) {
87         digitalWrite(dirPin1, LOW);
88         stepMotor(stepPin1, map(xVal, 489, 0, 2000, 800));
89         currentXSteps--;
90     }
91
92     // --- Process Motor 2 (Y-Axis) ---
93     // SLOWER PACE: Delay mapped from 4000 (slowest) to 1500 (fastest)
94     if (yVal > deadzoneHigh && currentYSteps < maxYSteps) {
95         digitalWrite(dirPin2, HIGH);
96         stepMotor(stepPin2, map(yVal, 531, 1023, 4000, 1500));
97         currentYSteps++;
98     } else if (yVal < deadzoneLow && currentYSteps > 0) {
```

```
99     digitalWrite(dirPin2, LOW);
100     stepMotor(stepPin2, map(yVal, 489, 0, 4000, 1500));
101     currentYSteps--;
102 }
103
104 // --- Update LCD Non-Blocking ---
105 if (millis() - lastLCDUpdate >= lcdInterval) {
106     lastLCDUpdate = millis();
107     updateLCD();
108 }
109 }
110
111 // --- Reusable Motor Stepping Function ---
112 void stepMotor(int pin, int speed) {
113     digitalWrite(pin, HIGH);
114     delayMicroseconds(speed);
115     digitalWrite(pin, LOW);
116     delayMicroseconds(speed);
117 }
118
119 // --- LCD Display Function ---
120 void updateLCD() {
121     float xMM = currentXSteps / stepsPerMM;
122     float yMM = currentYSteps / stepsPerMM;
123     lcd.setCursor(3, 0);
124     lcd.print(xMM, 1);
125     lcd.print(" mm ");
126     lcd.setCursor(3, 1);
127     lcd.print(yMM, 1);
128     lcd.print(" mm ");
129 }
130
131 // --- Return to Home Function ---
132 void returnToHome() {
133     // 1. Force the laser OFF for safety
134     laserState = LOW;
135     digitalWrite(laserPin, laserState);
136
137     // 2. Update LCD to show Homing Status
138     lcd.clear();
139     lcd.setCursor(0, 0);
140     lcd.print("Returning to");
141     lcd.setCursor(0, 1);
142     lcd.print("Home (0,0)...");
143
144     // 3. Set both motor directions to backward
145     digitalWrite(dirPin1, LOW);
146     digitalWrite(dirPin2, LOW);
147
148     // 4. Step both motors simultaneously until counters reach 0
```



```
149 while (currentXSteps > 0 || currentYSteps > 0) {
150     if (currentXSteps > 0) {
151         stepMotor(stepPin1, 800); // Safe homing speed for X
152         currentXSteps--;
153     }
154     if (currentYSteps > 0) {
155         stepMotor(stepPin2, 1500); // Safe homing speed for Y
156         currentYSteps--;
157     }
158 }
159
160 // 5. Clean up display and force reset to exactly 0.0
161 lcd.clear();
162 lcd.setCursor(0, 0);
163 lcd.print("X: 0.0 mm");
164 lcd.setCursor(0, 1);
165 lcd.print("Y: 0.0 mm");
166 delay(500);
167 }
```

Listing 1: Manual joystick control, LCD coordinate display, laser toggle, and reset-to-origin code.

B. Automated Control Arduino Code

```
1 #include <LiquidCrystal.h>
2
3 LiquidCrystal lcd(8, 9, 10, 11, 12, 13);
4
5 // --- MOTOR PIN SWAP ---
6 const int dirPinY = 2;
7 const int stepPinY = 3;
8 const int dirPinX = 5;
9 const int stepPinX = 6;
10
11 const int joyX = A0;
12 const int joyY = A1;
13 const int laserPin = 7;
14 const int joySW = 4;
15 const int resetBtnPin = A2;
16
17 // --- Adjusted Settings for Lead Screw Torque ---
18 const float stepsPerMM = 141.74;
19 const float stepsPerInch = stepsPerMM * 25.4;
20
21 // HIGHER number = SLOWER speed.
22 // Increased from 800/1500 to 3000/3500 to help the bed overcome friction.
23 const int autoSpeedX = 3000;
24 const int autoSpeedY = 3500;
25
26 enum State { MENU, DRAWING };
27 State currentState = MENU;
28 int menuSelection = 0;
29 const char* shapes[] = {"Square", "Hexagon", "Triangle", "Circle", "Star"};
30 const int totalShapes = 5;
31
32 long currentXSteps = 0;
33 long currentYSteps = 0;
34
35 void setup() {
36   lcd.begin(16, 2);
37   pinMode(dirPinX, OUTPUT); pinMode(stepPinX, OUTPUT);
38   pinMode(dirPinY, OUTPUT); pinMode(stepPinY, OUTPUT);
39   pinMode(laserPin, OUTPUT);
40   pinMode(joySW, INPUT_PULLUP);
41   pinMode(resetBtnPin, INPUT_PULLUP);
42   updateMenuDisplay();
43 }
44
45 void loop() {
46   if (digitalRead(resetBtnPin) == LOW) {
47     returnToHome();
48     currentState = MENU;
```

```
49     updateMenuDisplay();
50 }
51
52 if (currentState == MENU) {
53     handleMenu();
54 }
55 }
56
57 void handleMenu() {
58     int xVal = analogRead(joyX);
59     if (xVal > 800) {
60         menuSelection = (menuSelection + 1) % totalShapes;
61         updateMenuDisplay();
62         delay(300);
63     } else if (xVal < 200) {
64         menuSelection = (menuSelection - 1 + totalShapes) % totalShapes;
65         updateMenuDisplay();
66         delay(300);
67     }
68
69     if (digitalRead(joySW) == LOW) {
70         delay(500);
71         if (menuSelection == 0) {
72             currentState = DRAWING;
73             drawSquare();
74             currentState = MENU;
75             updateMenuDisplay();
76         }
77     }
78 }
79
80 void updateMenuDisplay() {
81     lcd.clear();
82     lcd.setCursor(0, 0);
83     lcd.print("Select Shape:");
84     lcd.setCursor(0, 1);
85     lcd.print("> ");
86     lcd.print(shapes[menuSelection]);
87 }
88
89 void moveRelative(long deltaX, long deltaY) {
90     digitalWrite(dirPinX, (deltaX >= 0) ? HIGH : LOW);
91     digitalWrite(dirPinY, (deltaY >= 0) ? HIGH : LOW);
92
93     long absX = abs(deltaX);
94     long absY = abs(deltaY);
95     long countX = 0;
96     long countY = 0;
97
98     // Move until both axes reach their destination
```

```
99   while (countX < absX || countY < absY) {
100       if (countX < absX) {
101           stepMotor(stepPinX, autoSpeedX);
102           countX++;
103           currentXSteps += (deltaX > 0) ? 1 : -1;
104       }
105       if (countY < absY) {
106           stepMotor(stepPinY, autoSpeedY);
107           countY++;
108           currentYSteps += (deltaY > 0) ? 1 : -1;
109       }
110   }
111 }
112
113 void drawSquare() {
114     lcd.clear();
115     lcd.print("Drawing...");
116
117     moveRelative(stepsPerMM * 10, stepsPerMM * 10); // Start offset
118     digitalWrite(laserPin, HIGH);
119     delay(200);
120
121     moveRelative(0, stepsPerInch);           // Side 1
122     moveRelative(stepsPerInch, 0);           // Side 2
123     moveRelative(0, -stepsPerInch);          // Side 3
124     moveRelative(-stepsPerInch, 0);          // Side 4
125
126     digitalWrite(laserPin, LOW);
127     delay(500);
128     returnToHome();
129 }
130
131 // Improved step function with longer pulse width
132 void stepMotor(int pin, int speed) {
133     digitalWrite(pin, HIGH);
134     delayMicroseconds(10); // Pulse width duration
135     digitalWrite(pin, LOW);
136     delayMicroseconds(speed); // Travel time (speed control)
137 }
138
139 void returnToHome() {
140     digitalWrite(laserPin, LOW);
141     lcd.clear();
142     lcd.print("Homing...");
143     digitalWrite(dirPinX, LOW);
144     digitalWrite(dirPinY, LOW);
145
146     while (currentXSteps > 0 || currentYSteps > 0) {
147         if (currentXSteps > 0) { stepMotor(stepPinX, autoSpeedX); currentXSteps--; }
148         if (currentYSteps > 0) { stepMotor(stepPinY, autoSpeedY); currentYSteps--; }
```

```
149     }  
150 }
```

Listing 2: Automated menu-driven control and square drawing routine.

References

- [1] Thomasnet, “Laser Engraving: Definition, Process, Materials, and Types,” 2025. [Online]. Available: <https://www.thomasnet.com/articles/machinery-tools-supplies/laser-engraving/>. Accessed: May 2026.
- [2] Arduino, “UNO R3 Documentation,” 2026. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/>. Accessed: May 2026.
- [3] Arduino, “Arduino and Stepper Motor Configurations,” 2022. [Online]. Available: <https://docs.arduino.cc/learn/electronics/stepper-motors/>. Accessed: May 2026.
- [4] Arduino, “LiquidCrystal Library Documentation,” 2026. [Online]. Available: <https://docs.arduino.cc/libraries/liquidcrystal/>. Accessed: May 2026.
- [5] Pololu, “A4988 Stepper Motor Driver Carrier,” 2026. [Online]. Available: <https://www.pololu.com/product/1182>. Accessed: May 2026.
- [6] Grbl Project, “Grbl: An Open Source Embedded G-code Parser and CNC Controller,” 2019. [Online]. Available: <https://github.com/grbl/grbl>. Accessed: May 2026.